



Object Oriented System Analysis

CHAPTER 6

What is object orientation?

- ❑ In the object-oriented approach, the focus is on capturing the structure and behavior of information systems into small modules that combines both data and process.
- ❑ main aim of Object Oriented Design (OOD) is to **improve** the **quality** and productivity of system analysis and design by making it more **usable**.
- ❑ In analysis phase, OO models are used to fill the gap between problem and solution.
- ❑ It performs well in **situation** where systems are undergoing **continuous design**, **adaption**, and **maintenance**.
- ❑ It **identifies** the **objects** in problem domain, classifying them in terms of data and behavior.

What is object orientation?...

- The OO model is beneficial in the following ways
 - It **facilitates changes** in the system at low cost.
 - It promotes the **reuse** of components.
 - It **simplifies** the problem of **integrating** components to configure large system.
 - It **simplifies** the **design** of distributed systems

What is object orientation?...

□ the characteristics of OO System

- ✓ **Objects** is something that exists within problem domain and can be identified by data (attribute) or behavior. All tangible entities (student, patient) and some intangible entities (bank account) are modeled as object.
- ✓ **Attributes** – They describe information about the object.
- ✓ **Behavior** – It specifies what the object can do. It defines the operation performed on objects.
- ✓ **Class** – A class encapsulates the data and its behavior. Objects with similar meaning and purpose grouped together as class.
- ✓ **Methods** – Methods determine the behavior of a class. They are nothing more than an **action** that an object can perform.
- ✓ **Message** – A message is a function or procedure **call** from one object to another. They are information sent to objects to trigger methods. Essentially, a message is a function or procedure call from one object to another.

What is object orientation?...

❑ Features of Object-Oriented System

✓ **Encapsulation**

- is a process of information hiding. It is simply the combination of process and data into a single entity.
- Data of an object is hidden from the rest of the system and available only through the services of the class.
- It allows improvement or modification of methods used by objects without affecting other parts of a system.

What is object orientation?...

❑ Features of Object-Oriented System

✓ **Abstraction**

- It is a process of taking or selecting necessary method and attributes to specify the object.
- It focuses on essential characteristics of an object relative to perspective of user.

What is object orientation?...

❑ Features of Object-Oriented System

✓ **Relationships**

- All the classes in the system are related with each other.
 - The objects do not exist in isolation, they exist in relationship with other objects
 - There are three types of object **relationships**
1. **Aggregation** – It indicates relationship between a whole and its parts.
 2. **Association** – In this, two classes are related or connected in some way such as **one class works with another to perform a task** or one class acts upon other class.
 3. **Generalization** – The child class is based on parent class. It indicates that two classes are similar but have some differences.

What is object orientation?...

❑ Features of Object-Oriented System

✓ **Inheritance**

- is a great feature that allows to create sub-classes from an existing class by inheriting the attributes and/or operations of existing classes.

✓ **Polymorphism and Dynamic Binding**

- is the ability to take on many different forms.
- It applies to both objects and operations.
- A polymorphic object is one whose true type hides within a super or parent class.
- In polymorphic operation, the operation may be carried out differently by different classes of objects.
- It allows us to manipulate objects of different classes by knowing only their common properties.

Model

- A model is a simplification at some level of abstraction
- We build models to better understand the systems we are developing.

Model...

□ Why do we model?

- Helps us to visualize a system as we want it to be.
- Permits us to specify the structure or behavior of a system.
- Gives us a template that guides us in constructing a system.
- To document decisions we have made

Principles of Modeling

- ❑ The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
 - Choose your models well. The right models will highlight the most nasty development problems. Wrong models will mislead you, causing you to focus on irrelevant issues.
- ❑ Every model may be expressed at different levels of precision.
 - Sometimes, a quick and simple executable model of the user interface is exactly what you need. At other times, you have to get down to complex details such as cross-system interfaces or networking issues etc.
 - In any case, the best kinds of models are those that let you choose your degree of detail, depending on who is viewing it. An analyst or an end user will want to focus on issues of what and a developer will want to focus on issues of how.

Principles of Modeling...

- ❑ The best models are connected to reality

- In software, the gap between the analysis model and the system's design model must be less. Failing to bridge this gap causes the system to diverge over time. In object-oriented systems, it is possible to connect all the nearly independent views of a system into one whole.

- ❑ No single model is sufficient. Every non-trivial system is best approached through a small set of nearly independent models.

Object oriented modeling

- ❑ Object-oriented modeling (OOM) is the construction of objects using a collection of objects that contain stored values of the instance variables found within an object.
- ❑ Unlike models that are record-oriented, object-oriented values are solely objects.
- ❑ The object-oriented modeling approach creates the union of the application and database development and transforms it into a unified data model and language environment.
- ❑ Object-oriented modeling allows for object identification and communication while supporting data abstraction, inheritance and encapsulation

Object oriented modeling...

- ❑ The software life cycle is typically divided up into stages going from abstract descriptions of the problem to designs then to code and testing and finally to deployment.
- ❑ Modeling is done at the beginning of the process.
- ❑ The **reasons to model** a system before writing the code are:
 - **Communication:** Users typically cannot understand programming language or code.
 - Model diagrams can be more understandable and can allow users to give developers feedback on the appropriate structure of the system.
 - A key goal of the Object-Oriented approach is to decrease the "semantic gap" between the system and the real world by using terminology that is the same as the functions that users perform

Object oriented modeling...

- ❑ The reasons to model a system before writing the code are:
 - **Abstraction:** goal of most software methodologies is to first address "what" questions and then address "how" questions. I.e., **first determine the functionality** the system is to provide **without consideration of implementation constraints** and **then** consider how to take this abstract description and **refine it** into an **implementable design** and code given constraints such as technology and budget.
 - Modeling enables this by allowing abstract descriptions of processes and objects that define their essential structure and behavior.
- ❑ Object-oriented modeling is typically done via use case and abstract definitions of the most important objects.
- ❑ The **most common language** used **to do object-oriented modeling** is the Object management Group's Unified Modeling Language (UML)

Overview of UML

- ❑ UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.
- ❑ UML stands for **Unified Modeling Language**.
- ❑ UML is different from the other common programming languages such as C++, Java, COBOL, etc.
- UML is a **pictorial language** used to make software blueprints.
- UML can be described as a **general purpose visual modeling language** to visualize, specify, construct, and document software system.
- Although UML is generally used to model software systems, it is **not limited within this boundary**. It is also used to model non-software systems as well. For example, the process flow in a manufacturing unit, etc.

Overview of UML...

□ Goals of UML

- Provide users with a ready-to-use, expressive visual modeling language so they can develop and exchange meaningful models
- Provide extensibility and specialization mechanisms to extend the core concepts
- Be independent of particular programming languages and development processes
- Provide a formal basis for understanding the modeling language
- Encourage the growth of the OO tools market
- Support higher-level development concepts such as collaborations, frameworks, patterns and components
- Integrate best practices

Overview of UML...

- ❑ **Building Blocks of the UML**

- ❑ The vocabulary of the UML encompasses three kinds of building blocks

- 1. Things**

- 2. Relationships**

- 3. Diagrams**

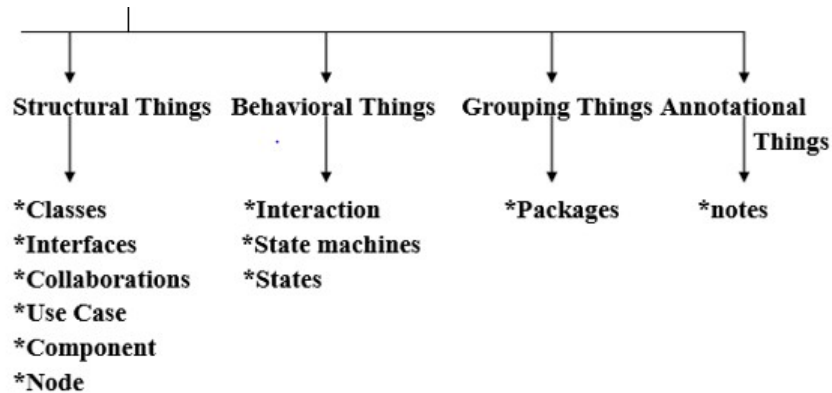
Overview of UML...

□ Things

- Things are the abstractions that are first-class citizens in a model; relationships tie these things together; diagrams group interesting collections of things.
- There are 4 kinds of things in the UML:
 - Structural things
 - Behavioral things
 - Grouping things
 - Annotational things
- These things are the basic object-oriented building blocks of the UML. You use them to write well-formed models.

Overview of UML...

□ Things



Overview of UML...

□ **Structural things** are the **nouns of UML models**.

- These are the mostly static parts of a model, representing elements that are either conceptual or physical.
- Collectively, the structural things are called classifiers.

Overview of UML...

□ **Behavioral things** are the dynamic parts of UML models.

- These are the verbs of a model, representing behavior over time and space.

□ **Grouping things** are the **organizational parts of UML models**.

- These are the boxes into which a model can be decomposed. There is one primary kind of grouping thing, namely, packages.

Overview of UML...

- ❑ **Annotational things** are the **explanatory parts of UML** models.
 - These are the comments you may apply to describe, illuminate, and remark about any element in a model.
 - There is one primary kind of annotational thing, called a **note**.
 - A note is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.
- ❑ Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment

Overview of UML...

□ Relationships

□ There are 4 kinds of relationships in the UML:

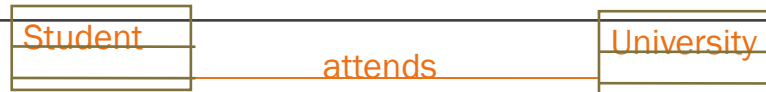
- Dependency
- Association
- Generalization
- Realization

□ These relationships are the basic relational building blocks of the UML

Overview of UML...

1. Associations

Structural relationship that describes a set of links, a link being a connection between objects. variants: *aggregation & composition*



2. Generalization

a specialized element (the child) is more specific the generalized element.



3. Realization

one element guarantees to carry out what is expected by the other element.
(e.g. interfaces and classes/components; use cases and collaborations)



4. Dependency

a change to one thing (independent) may affect the semantics of the other thing (dependent).
(direction, label are optional)



Overview of UML...

❑ Diagrams

- ❑ It is the graphical presentation of a set of elements. It is rendered as a connected graph of vertices (things) and arcs (relationships).
- ❑ In theory, a diagram may contain any **combination of things and relationships**.
 - Class diagram
 - Object diagram
 - Use case diagram
 - Sequence diagram
 - Collaboration diagram
 - State chart diagram
 - Activity diagram
 - Component diagram
 - Deployment diagram

Overview of UML...

❑ Rules:

The UML has a number of rules that specify what a well-formed model should look like.

❑ A well-formed model is one that is semantically self-consistent and in harmony with all its related models.

❑ The UML has semantic rules for:

- **Names** – What you can call things, relationships, and diagrams.
- **Scope** – The context that gives specific meaning to a name.
- **Visibility** – How those names can be seen and used by others.
- **Integrity** – How things properly and consistently relate to one another.

Use Case Diagrams

□ What is a Use Case

- A formal way of representing **how a business system interacts with its environment**
 - Illustrates the **activities** that are performed by the users of the system
- A scenario-based technique in the UML

Use Case ...

□ What is an **Actor**?

- A **user** or **outside system** that interacts with the system being designed in order to obtain some value from that interaction
- **Use Cases** describe **scenarios that describe the interaction between users of the system (the actor) and the system itself.**

Use Cases ...

- **Use case diagrams** describe what a system does from the standpoint of an external observer.
- The emphasis is on *what* a system does rather than *how*.
- Use case diagrams are closely connected to scenarios.
- A **scenario** is an example of what happens when someone interacts with the system.

Use Cases ...

- Here is a scenario for a medical clinic.

□ *A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot. "*

- We want to write a use case for this scenario.

□ Remember: A **use case** is a summary of scenarios for a single task or goal.

Use Cases ...

- ❑ Step 1 Identify the actors
- ❑ Define those people or systems that are going to interact with the scenario.
- ❑ *A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot. "*

Use Cases ...

Questions for Identifying People Actors

- ☐ Who is interested in the scenario/system?
- ☐ Where in the organization is the scenario/system be used?
- ☐ Who will benefit from the use of the scenario/system?
- ☐ Who will supply the scenario/system with this information, use this information, and remove this information?
- ☐ Does one person play several different roles?
- ☐ Do several people play the same role?

Use Cases ...

Questions for Identifying People Actors

- ☐ What other entity is interested in the scenario/system?
- ☐ What other entity will supply the scenario/system with this information, use this information, and remove this information?
- ☐ Does the system use an external resource?
- ☐ Does the system interact with a legacy system?

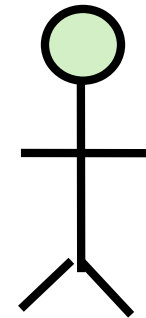
Actors

❑ An Actor is outside or external the system.

❑ It can be a:

- Human
- Peripheral device (hardware)
- External system or subsystem
- Time or time-based event

❑ Represented by stick figure



Use Cases

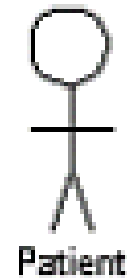
- ❑ A **use case** is a **summary of scenarios** for a **single task or goal**.
- ❑ An **actor** is who or what **initiates the events** involved in the task of the use case. Actors are simply roles that people or objects play.
- ❑ So as we read our scenario, what or who is the actor????

Use Cases

❑ So as we read our scenario, what or who is the actor????

❑ *A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot. "*

❑ The actor is a **Patient**.

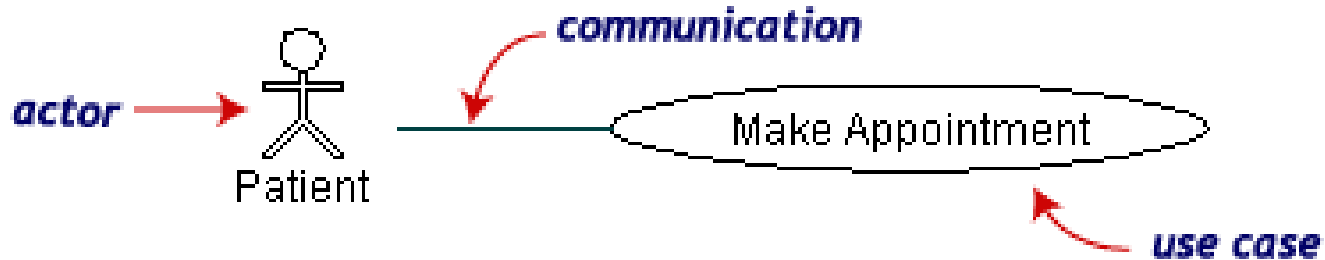


Use Cases

- ❑ The **use case** is a summary of scenarios for a single task or goal.
- ❑ So What is the Use Case????
- ❑ The Use Case is **Make Appointment**.
- ❑ It is a use case for the medical clinic.

Use Cases

- ❑ The picture below is a **Make Appointment** use case for the medical clinic.
- ❑ The actor is a **Patient**. The connection between actor and use case is a **communication association** (or **communication** for short).



Actors are stick figures.

Use cases are ovals.

Communications are lines that **link actors to use cases**.

Use Case Components

- ❑ The use case has three components.
- ❑ The **use case** task referred to as the use case that represents a feature needed in a software system.
- ❑ The **actor(s)** who trigger the use case to activate.
- ❑ The **communication** line to show how the actors communicate with the use case.

Use Case Diagram - Use Case

- ❑ A major process performed by the system that benefits an actor(s) in some way
- ❑ Models a dialogue between an actor and the system
- ❑ Represents the functionality provided by the system

Use Case

□ Each use case in a use case diagram describes one and only one *function* in which users interact with the system

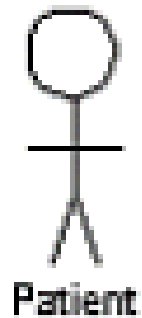
- May contain several “paths” that a user can take while interacting with the system
- Each path is referred to as a scenario

Use Case

- ❑ Labelled using a descriptive **verb-noun** phrase
- ❑ Represented by an oval

Use Case - Actor

- ❑ Labelled using a descriptive noun or phrase
- ❑ Represented by a stick character



Use Case - Relationships

□ Relationships

- Represent communication between actor and use case
- Depicted by line or double-headed arrow line



- Also called association relationship

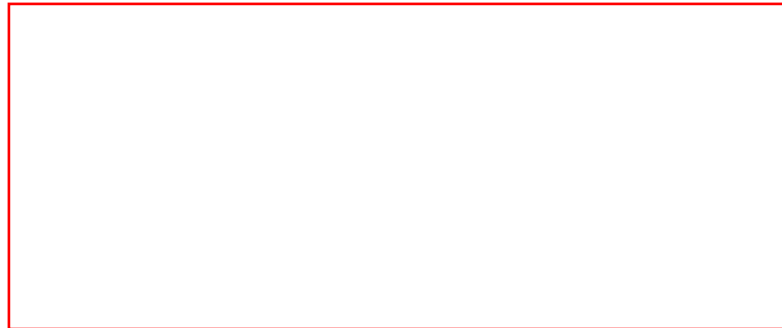
• Other Relationship

- Generalization
- Include
- Extend

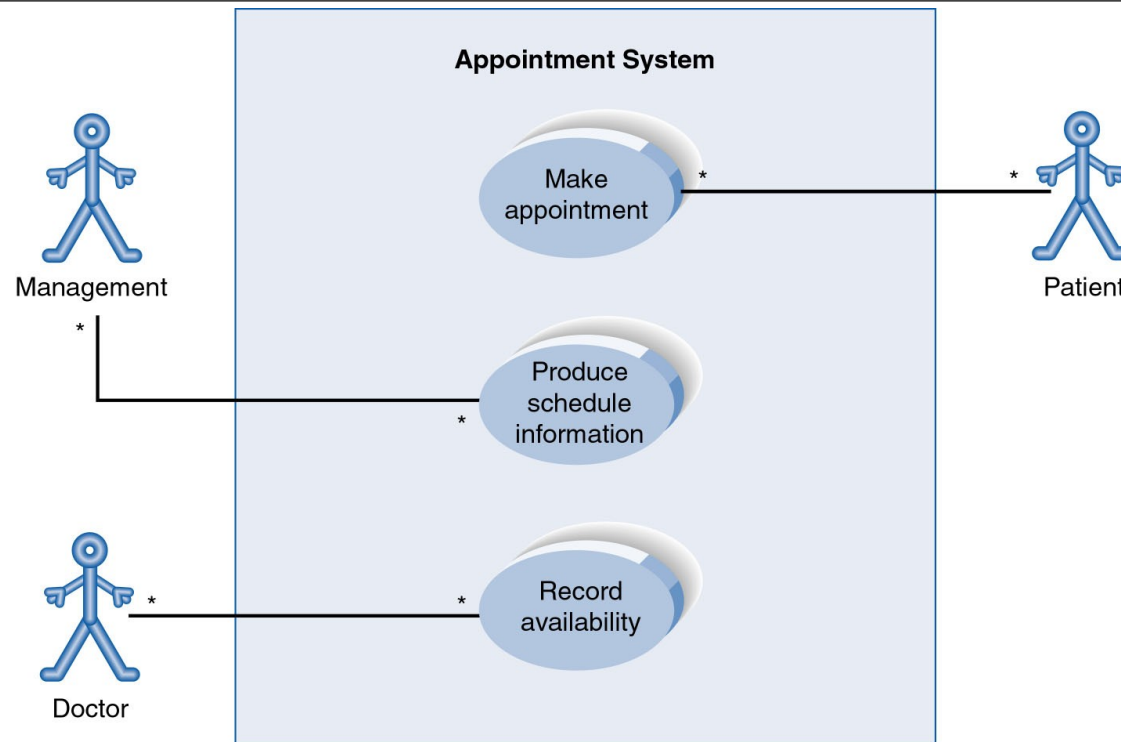
Use Case - Relationships

□ Boundary

- defines the scope of what a system will be
- A boundary rectangle is placed around the perimeter of the system to show how the actors communicate with the system.



Use-Case Diagram



A use case diagram is a collection of actors, use cases, and their communications.

Use Case Diagram

❑ Other Types of Relationships for Use Cases

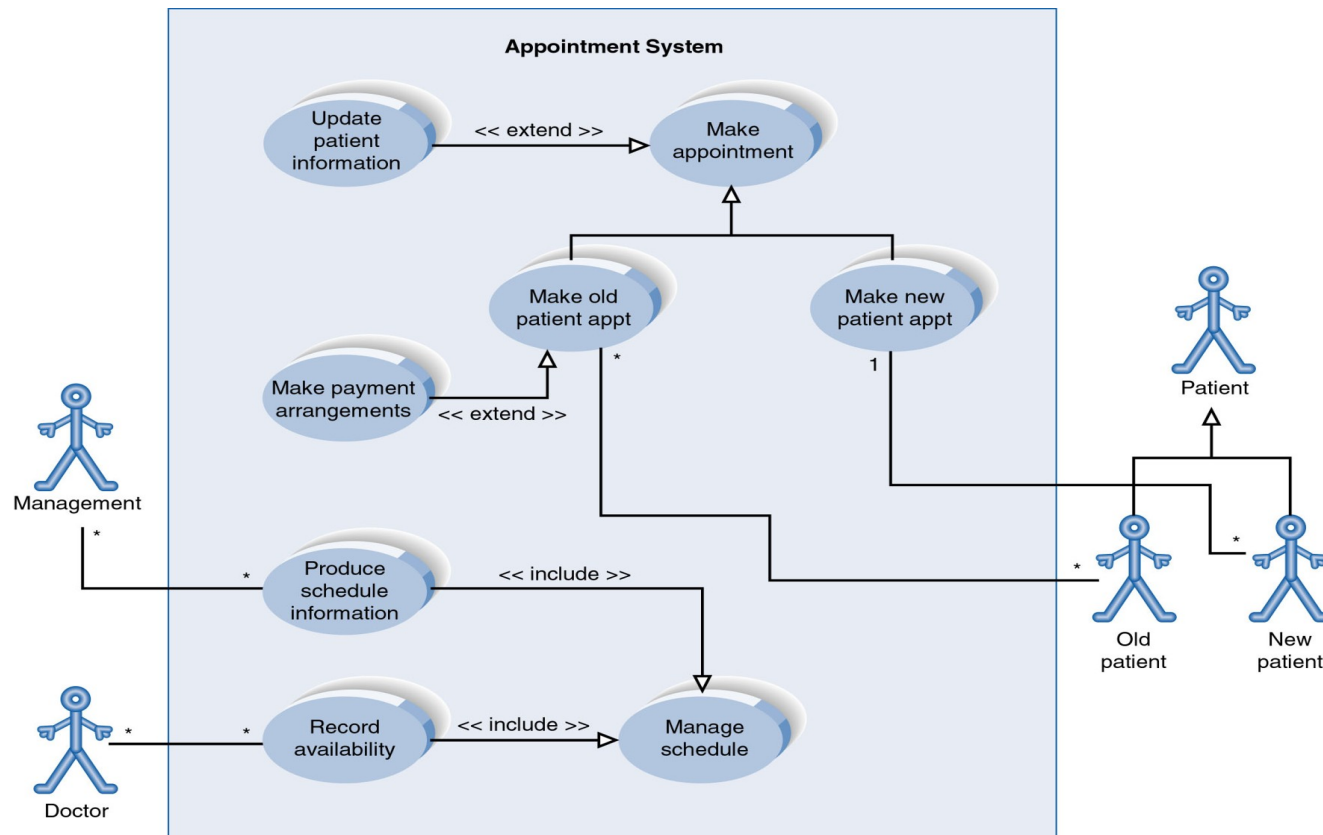
- Generalization
- Include
- Extend

Components of Use Case Diagram

□ Generalization Relationship

- Represented by a line and a hollow arrow
 - From child to parent

Example of Relationships



Use Case Diagram

□ Include Relationship

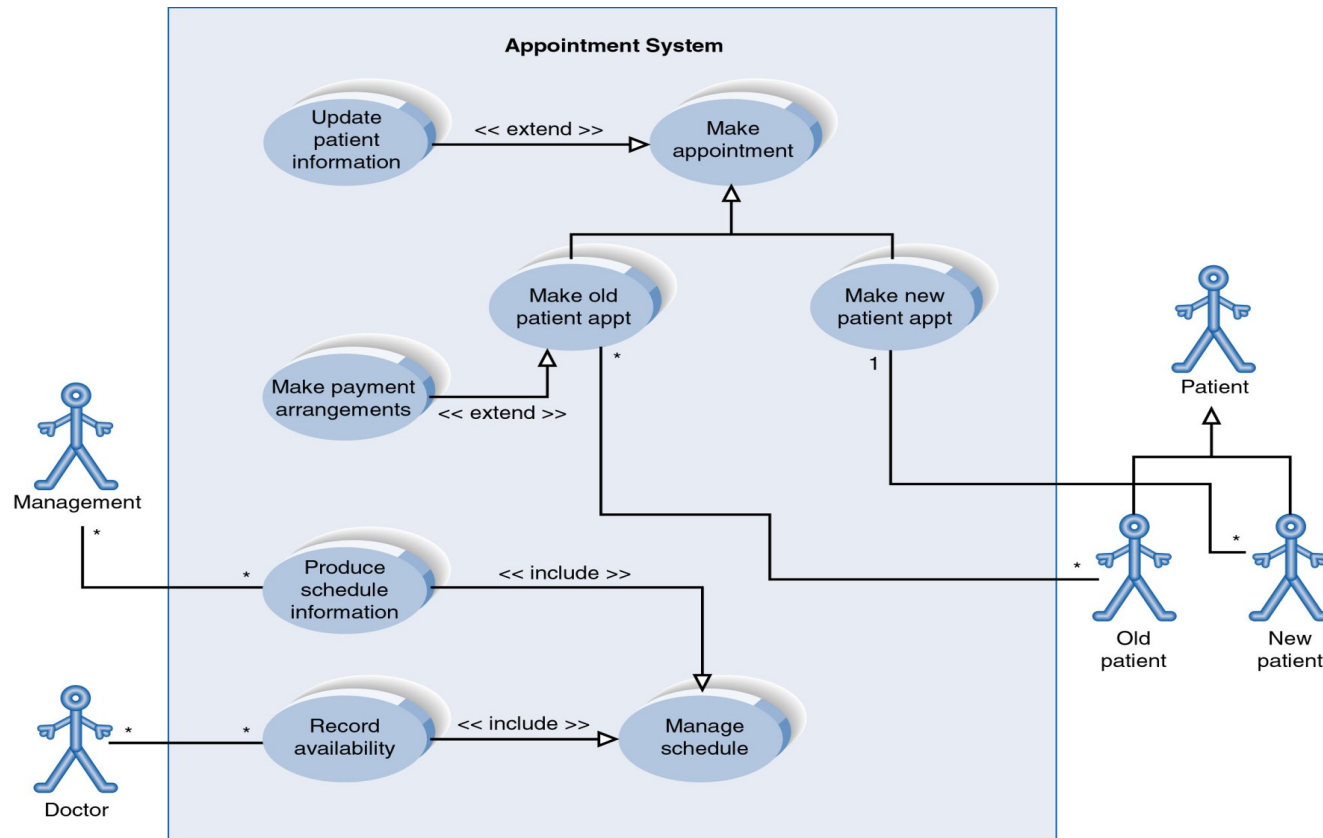
- Represents the inclusion of the functionality of one use case within another
- Arrow is drawn from the base use case to the used use case
- Write << include >> above arrowhead line

Use Case Diagram

□ Extend relationship

- Represents the extension of the use case to include optional functionality
- Arrow is drawn from the extension use case to the base use case
- Write << extend >> above arrowhead line

Example of Relationships



Use Cases Identification Techniques

❑ Three techniques for Identifying use cases

- User goal technique
- Event decomposition technique
- CRUD

❑ Name each use case using *Verb-Noun*

1. User Goal Technique

- ❑ This technique is the most common in industry
- ❑ Simple and effective
- ❑ Identify all of the potential categories of users of the system
- ❑ Interview and ask them to describe the tasks the system can help them with
 - E.g, “I need to *Ship items, Track a shipment, Create a return*”

1. User Goal Technique

Example

User	User goal and resulting use case
Potential customer	Search for item Fill shopping cart View product rating and comments
Marketing manager	Add/update product information Add/update promotion Produce sales history report
Shipping personnel	Ship items Track shipment Create item return

1. User Goal Technique: Specific Steps

- 1. Identify all the potential users** for the new system
- 2. Classify the potential users in terms of their functional role** (e.g., shipping, marketing, sales)
- 3. Further classify potential users by organizational level** (e.g., operational, management, executive)
- 4. For each type of user, interview them to find a list of specific goals** they will have when using the new system (current goals and innovative functions to add value)

1. User Goal Technique

Specific Steps ...

- 5. **Create a list of preliminary use cases organized by type of user**
- 5. **Look for duplicates with similar use case names and resolve inconsistencies**
- 5. Identify where different types of users need the same use cases
- 5. Review the completed list with each type of user and then with interested stakeholders

2. Event Decomposition Technique

- ❑ More Comprehensive and Complete Technique
 - **Identify the events that occur to which the system must respond.**
 - For each event, name a use case (verb-noun) that describes what the system does when the event occurs
- ❑ Event— **something that occurs at a specific time and place**, can be described, and should be remembered by the system

Types of Events

❑ External Event

- an event that occurs outside the system, usually initiated by an external agent or actor

❑ Temporal Event

- an event that occurs as a result of reaching a point in time

❑ State Event

- an event that occurs when something happens inside the system that triggers some process
- reorder point is reached for inventory item

External Event Checklist

- ☐ External agent or actor wants something resulting in a transaction
 - Customer buys a product
- ☐ External agent or actor wants some information
 - Customer wants to know product details
- ☐ External data changed and needs to be updated
 - Customer has new address and phone
- ☐ Management wants some information
 - Sales manager wants update on production plans

Temporal Event Checklist

- ❑ Internal outputs needed at points in time
 - Management reports (summary or exception)
 - Operational reports (detailed transactions)
 - Internal statements and documents (including payroll)
- ❑ External outputs needed at points of time
 - Statements, status reports, bills, reminders

Event Decomposition Technique: Specific Steps

1. Consider the **external events** in the system environment that **require a response from the system** by using the checklist shown in
2. For each external event, **identify and name the use case that the system requires**
3. Consider the **temporal events** that **require a response from the system** by using the checklist
4. For each temporal event, **identify and name the use case that the system requires** and then **establish the point of time that will trigger the use case**

Event Decomposition Technique: Specific Steps ...

5. **Consider the state events that the system might respond to**, particularly if it is a real-time system in which devices or internal state changes trigger use cases.
5. For each state event, identify and name the use case that the system requires and then define the state change.
5. When events and use cases are defined, check to see if they are required by using the perfect technology assumption.
 - Do not include **events** that involve such **system controls** as login, logout, change password, and backup or restore the database, as these are put in later.

Event Decomposition Technique: Benefits

- ❑ Events are broader than user goal: Capture temporal and state events
- ❑ Help decompose at the right level of analysis: an elementary business process (EBP)
- ❑ EBP is a fundamental business process performed by one person, in one place, in response to a business event
- ❑ Uses perfect technology assumption to make sure **functions that support the users work are identified** and not additional functions for security and system controls

The
END